



Guia de Dúvidas e Conceitos

Sistema de Controle de Leilões

FAQ Completo — Programação de Software Aplicado | 2026/1

Junho de 2026



Arquitetura e Padrões de Projeto

14 perguntas

1. O que é arquitetura multicamada e por que usamos?

É a divisão do sistema em camadas com responsabilidades bem definidas: **Apresentação** (Controllers + Thymeleaf), **Serviço** (regras de negócio), **Domínio** (entidades JPA) e **Persistência** (Repositories). Cada camada só se comunica com a imediatamente abaixo. Isso facilita manutenção, testes e permite trocar tecnologias sem reescrever o sistema inteiro.

2. O que é o padrão MVC e onde ele aparece no projeto?

MVC separa a aplicação em três partes: **Model** (DTOs/entidades com os dados), **View** (templates Thymeleaf em `src/main/resources/templates/`) e **Controller** (classes em `com.leiloes.controller` que recebem requisições HTTP, chamam os serviços e escolhem qual view renderizar).

3. O que é o padrão Domain Model e onde ele aparece?

O Domain Model representa os conceitos do negócio como classes Java ricas — objetos com identidade, relacionamentos e comportamento. No projeto estão em `com.leiloes.domain.model`: `Leilao`, `Lote`, `Lance`, `Bem`, `Usuario` e `Categoria`, todos mapeados via JPA/Hibernate para o banco PostgreSQL.

4. O que é o padrão Repository e onde ele aparece?

O Repository encapsula toda a lógica de acesso ao banco atrás de uma interface Java. Em `com.leiloes.repository`, cada entidade tem seu repositório (ex: `LeilaoRepository`) que estende `JpaRepository` — fornecendo CRUD automaticamente. O serviço nunca escreve SQL diretamente; sempre chama o repositório.

5. O que é o Specification Pattern e por que foi escolhido?

Encapsula uma regra de negócio em uma classe separada com o método `isSatisfiedBy(objeto)`. Em vez de acumular vários `if` no serviço, cada regra vira uma classe: `LeilaoEmAndamentoSpec`, `LanceAcimaDoMinimoSpec`, `LanceMaiorQueLanceAtualSpec` etc. Facilita adicionar novas regras sem modificar código existente e permite testar cada regra isoladamente.

6. Qual é o fluxo completo de uma requisição no sistema?

Usuário age no browser → **Controller** recebe o HTTP → chama o **Service** com DTO de input → Service valida via **Specifications** e chama o **Repository** → Repository executa query no **PostgreSQL** → dados voltam como entidades → Service converte em DTOs de output → Controller adiciona ao Model → **Thymeleaf** renderiza o HTML → Browser exibe.

7. Por que usamos DTOs em vez de passar entidades diretamente para a View?

As entidades JPA são objetos complexos que podem causar queries extras indesejadas (problema N+1) ao serem acessadas na View. Os DTOs (`LeilaoResumo`, `LeilaoDetalhe`) contêm exatamente os dados que cada tela precisa — sem dados extras — e são imutáveis (Java Records), garantindo que os dados não sejam modificados após sair da camada de serviço.

8. O que é o problema N+1 e como evitamos no projeto?

Ocorre quando, para exibir uma lista de N registros, o JPA executa 1 query para buscar a lista + N queries para cada relacionamento acessado. Evitamos usando `@Query` com `SELECT NEW` (JPQL) nos repositórios, que busca apenas os campos necessários e os mapeia direto em DTOs em uma única query.

9. O que é DTO e qual a diferença entre Input e Output?

DTO Input (ex: `LoteInput`, `LanceInput`): dados vindos do formulário. Contém validações (`@NotBlank`, `@Positive`) e usa IDs para relacionamentos. **DTO Output** (ex: `LeilaoResumo`, `LeilaoDetalhe`): dados do sistema indo para a tela. Contém nomes legíveis, valores formatados e informações calculadas.

10. Como o Spring Boot sabe em qual porta rodar e qual banco conectar?

Tudo é configurado em `src/main/resources/application.properties`: URL do banco (`spring.datasource.url`), usuário e senha do PostgreSQL, configurações do Hibernate, porta do servidor (`server.port`) e configurações do Thymeleaf. O Spring Boot lê esse arquivo automaticamente na inicialização.

11. O que é injeção de dependência e como funciona no projeto?

Princípio onde um objeto não cria suas dependências — o Spring as injeta automaticamente. Os Controladores recebem Serviços e Repositórios via construtor (sem `new`). O Spring detecta classes anotadas com `@Service`, `@Repository` e `@Controller`, cria as instâncias e injeta onde forem necessárias, desacoplando as classes.

12. O que é o `GlobalExceptionHandler` e para que serve?

Classe anotada com `@ControllerAdvice` que intercepta exceções lançadas em qualquer Controller. Sem ele, erros mostrariam a página de erro genérica do Spring. Com ele, capturamos `LeilaoNaoEncontradoException`, `LanceInvalidoException` etc. e redirecionamos para páginas HTML amigáveis com mensagem clara.

13. Por que as exceções estendem `RuntimeException` e não `Exception`?

`RuntimeException` é uma unchecked exception — o Java não obriga capturá-la com try/catch em todo lugar. Isso simplifica o código: os serviços lançam exceções livremente e o `GlobalExceptionHandler` captura centralizadamente. Com `Exception` (checked), cada método intermediário seria obrigado a declarar `throws`, poluindo o código.

14. O que é `@Transactional` e por que usamos nos serviços?

Garante que todas as operações de banco dentro do método ocorram em uma única transação ACID. Se qualquer passo falhar, tudo é revertido (rollback) automaticamente. No `LanceService.registrar()`, por exemplo, se o salvamento do Lance falhar, nenhuma mudança persiste no banco.



Spring Boot e Java

16 perguntas

15. O que é o Spring Boot e por que ele facilita o desenvolvimento?

Framework que elimina configuração manual do Spring tradicional. O servidor embutido (Tomcat) sobe automaticamente, as dependências do `pom.xml` são configuradas com valores padrão inteligentes (auto-configuration), e a aplicação inicia com `mvn spring-boot:run`. Não precisamos criar arquivos XML de configuração.

16. O que é o pom.xml e para que serve cada dependência?

Arquivo de configuração do Maven. Cada `<dependency>` baixa uma biblioteca: **spring-boot-starter-web** (servidor HTTP + MVC), **spring-boot-starter-data-jpa** (JPA + Hibernate), **spring-boot-starter-thymeleaf** (motor de templates), **postgresql** (driver JDBC), **spring-boot-starter-validation** (Bean Validation para DTOs).

17. O que é um Java Record e por que é ideal para DTOs?

Introduzido no Java 14+, um `record` é uma classe imutável e compacta. `record LoteResumo(Long id, String nome, int qtdBens)` gera automaticamente: construtor com todos os campos, getters, `equals()`, `hashCode()` e `toString()`. Perfeito para DTOs pois dados transferidos não devem ser modificados após criados.

18. O que é a Stream API e como usamos no projeto?

Permite processar coleções de forma declarativa. No projeto: `lista.stream().map(leilao -> new LeilaoResumo(...)).toList()` converte listas de entidades em listas de DTOs. `.stream()` cria o pipeline, `.map()` transforma cada elemento, `.toList()` coleta o resultado. Mais legível e conciso que um loop `for`.

19. O que é enum e como usamos TipoLeilao e StatusLeilao?

Um `enum` é um tipo com conjunto fixo de valores nomeados. `TipoLeilao` tem ABERTO e FECHADO; `StatusLeilao` tem AGUARDANDO, EM_ANDAMENTO e ENCERRADO. O JPA armazena o nome como String no banco (`@Enumerated(EnumType.STRING)`). Evita magic strings no código e permite comparações seguras com `==`.

20. O que é a palavra-chave "super" usada nas exceções customizadas?

Chama o construtor da classe pai. Em `LeilaoNaoEncontradoException`, `super(mensagem)` chama o construtor de `RuntimeException`, que armazena a mensagem internamente. Assim, `e.getMessage()` retorna o texto correto e os logs/stacktraces funcionam de forma padronizada.

21. O que é @Controller vs @RestController?

`@Controller`: o método retorna uma String com o nome do template Thymeleaf a renderizar (ex: `"leiloes/lista"`).
`@RestController`: combina `@Controller` + `@ResponseBody`, o método retorna dados (objeto Java) convertidos automaticamente para JSON e escritos no corpo da resposta HTTP. Usado em APIs REST, não em aplicações MVC com templates.

22. O que é @RequestMapping, @GetMapping e @PostMapping?

`@RequestMapping("/leiloes")` na classe define o prefixo base da rota. `@GetMapping` mapeia requisições HTTP GET (exibir páginas). `@PostMapping` mapeia requisições HTTP POST (processar formulários). Exemplo:

`@GetMapping("/{id}")` mapeia `GET /leiloes/123` e `@PostMapping("/novo")` mapeia `POST /leiloes/novo`.

23. O que é @PathVariable e @ModelAttribute?

`@PathVariable` : extrai valores da URL. Em `@GetMapping("/{id}")`, o parâmetro `@PathVariable Long id` captura o número da URL (ex: `/leiloes/42` → `id = 42`). `@ModelAttribute` : o Spring lê os campos do formulário HTML submetido e preenche automaticamente o objeto Java correspondente (ex: `LoteInput input` recebe todos os campos do form).

24. O que é BindingResult e como funciona a validação de formulários?

Com `@Valid` antes de um parâmetro, o Spring valida as anotações (`@NotBlank`, `@Positive` etc.) automaticamente. O `BindingResult` deve vir imediatamente após e armazena os resultados. Se `bindingResult.hasErrors()` for true, retornamos o formulário com erros; caso contrário, prosseguimos o fluxo normal.

25. O que é "Model" nos Controllers e como funciona?

`Model` é um mapa (chave → valor) que o Controller usa para passar dados para a View Thymeleaf.

`model.addAttribute("leiloes", lista)` disponibiliza a lista no template como `${leiloes}`. O Spring cria e gerencia esse objeto para cada requisição. No template, todos os atributos ficam acessíveis via `${nomeDaChave}`.

26. O que é "redirect:/leiloes" retornado por um Controller?

É o Redirect Pattern (PRG — Post/Redirect/Get). Após um `POST` bem-sucedido, em vez de retornar uma view diretamente, retornamos `"redirect:/leiloes"`. Isso instrui o browser a fazer um novo `GET /leiloes`, evitando que o usuário reenvie o formulário ao pressionar F5 (o que duplicaria o registro no banco).

27. O que é o Maven e o ciclo mvn spring-boot:run?

Maven é o gerenciador de build e dependências. O `pom.xml` descreve as dependências e como construir o projeto. `mvn spring-boot:run` compila o código, baixa as dependências e inicia o Tomcat embutido. O projeto fica disponível na porta configurada em `http://localhost:8080`.

28. Como o Spring sabe quais classes são componentes gerenciáveis?

Pelo Component Scanning: o Spring varre o pacote base (`com.leiloes`) procurando anotações estereótipo: `@Service` (camada de serviço), `@Repository` (persistência), `@Controller` (apresentação). Cada uma registra a classe como um Bean no contexto Spring, tornando-a disponível para injeção automática.

29. O que é Bean Validation e quais anotações usamos?

Especificação Java para validação declarativa. Nos DTOs Input: `@NotBlank` (String não pode ser nula/vazia), `@NotNull` (campo obrigatório), `@Positive` (número maior que zero), `@Size` (tamanho de strings/coleções), `@Future` (datas futuras). O Spring ativa a validação com `@Valid` no parâmetro do Controller.

30. O que é Optional e por que usamos .orElseThrow() nos repositórios?

`Optional<T>` é um container que pode ou não conter um valor, eliminando `NullPointerException`.

`findById(id)` retorna `Optional<Entidade>`. Usamos `.orElseThrow(() -> new`

`LeilaoNaoEncontradoException("..."))` : se existir retorna a entidade; se não, lança a exceção customizada que o `GlobalExceptionHandler` captura.



JPA, Hibernate e Banco de Dados

14 perguntas

31. O que é JPA e o que é Hibernate?

JPA (Jakarta Persistence API) é uma especificação (interfaces e regras) para mapeamento objeto-relacional em Java.

Hibernate é a implementação mais popular do JPA — é ele quem gera e executa as queries SQL no banco. O Spring Data JPA é uma camada adicional que simplifica ainda mais, gerando automaticamente os métodos CRUD a partir das interfaces Repository.

32. O que fazem as anotações @Entity, @Table, @Id, @GeneratedValue?

`@Entity` : marca a classe como mapeada para uma tabela. `@Table(name="leiloes")` : define o nome da tabela no banco. `@Id` : marca o campo como chave primária. `@GeneratedValue(strategy = GenerationType.IDENTITY)` : o banco gera o ID automaticamente (SERIAL no PostgreSQL). Sem essas anotações, o JPA não sabe mapear a classe.

33. O que é @Column e quando precisamos usá-la?

`@Column` personaliza o mapeamento campo→coluna. Usamos quando o nome Java é diferente do nome no banco (`@Column(name = "data_inicio")`), para restrições (`nullable = false` , `unique = true` , `length = 200`). Sem ela, o JPA assume que o nome do campo Java é igual ao nome da coluna.

34. O que são @ManyToOne, @OneToMany e @ManyToMany?

Anotações para relacionamentos entre entidades. `@ManyToOne` : muitos Lances pertencem a um Leilão. `@OneToMany` : um Lote tem vários Bens. `@ManyToMany` : muitos Lotes podem ter muitos Bens. O JPA usa essas anotações para gerar os JOINS corretos e gerenciar as chaves estrangeiras no banco.

35. O que é FetchType.LAZY vs EAGER?

`LAZY` : o relacionamento só é carregado quando você o acessa pela primeira vez. Evita queries desnecessárias. `EAGER` : o relacionamento é carregado imediatamente junto com a entidade principal, sempre. Usamos LAZY por padrão (default para `@OneToMany` e `@ManyToMany`) para melhorar performance.

36. O que é a anotação @Query e quando ela é necessária?

Permite escrever consultas customizadas em JPQL ou SQL nativo. Usamos quando: (1) precisamos de dados de múltiplas entidades com JOIN mapeados em DTOs; (2) métodos automáticos do Spring Data não são suficientes; (3) precisamos de agregações (`COUNT` , `MAX`); (4) `nativeQuery = true` para SQL específico do PostgreSQL (`LIMIT` , `ORDER BY`).

37. O que é JPQL e como diferencia de SQL?

JPQL (Jakarta Persistence Query Language) é orientado a objetos: referencia classes Java e seus campos, não tabelas e colunas. SQL: `SELECT * FROM leiloes WHERE status = 'EM_ANDAMENTO'` . JPQL equivalente: `SELECT l FROM Leilao l WHERE l.status = :status` . O Hibernate traduz para o SQL correto do banco automaticamente.

38. O que é SELECT NEW no JPQL e por que usamos?

`SELECT NEW com.leiloes.dto.LeilaoResumo(l.id, l.nome, COUNT(la))` instancia diretamente o DTO dentro da query JPQL, sem criar a entidade completa. O banco retorna apenas os campos necessários, sem carregar entidades inteiras na memória. Evita o N+1 e reduz o consumo de memória e tempo de resposta.

39. O que é uma Projection e quando usamos?

Interface Java (ex: `LanceVencedorProjection`) com métodos getter. O Spring Data JPA mapeia automaticamente o resultado de uma query nativa para os métodos da interface. Usamos quando precisamos de SQL nativo complexo (`ORDER BY valor DESC LIMIT 1` para o lance vencedor) onde o JPQL não é suficiente.

40. O que é o schema.sql e como o Spring Boot o usa?

Arquivo com comandos `CREATE TABLE` que definem a estrutura do banco PostgreSQL. O Spring Boot pode executá-lo automaticamente na inicialização se configurado. Define: nomes de tabelas, colunas, tipos de dados, chaves primárias (PRIMARY KEY), chaves estrangeiras (FOREIGN KEY) e restrições de unicidade (UNIQUE).

41. O que é o seed.sql e por que é importante?

Contém comandos `INSERT INTO` com dados de teste fictícios mas realistas: usuários, categorias, bens, lotes e leilões em diferentes estados (aguardando, em andamento, encerrado) com lances registrados. Sem esses dados, o sistema iniciaria vazio e seria impossível testar as funcionalidades sem cadastrar tudo manualmente.

42. Como identificar os IDs corretos ao escrever o seed.sql?

As PKs são geradas sequencialmente (SERIAL começando em 1). O 1º INSERT de usuário ganha ID 1, o 2º ganha ID 2, etc. Para relacionamentos: se uma categoria foi o 3º INSERT, tem ID 3. Ao inserir um Bem nessa categoria, use `categoria_id = 3`. Leia o script de cima para baixo contando a ordem de inserção.

43. O que é SERIAL e como funciona no PostgreSQL?

`SERIAL` cria uma coluna inteira autoincrementada. Cada `INSERT` sem especificar o ID atribui automaticamente o próximo número da sequência. No mapeamento JPA, corresponde ao `@GeneratedValue(strategy = GenerationType.IDENTITY)`.

44. O que são chaves primárias (PK) e chaves estrangeiras (FK)?

PK (Primary Key): identifica unicamente cada linha de uma tabela. Não pode ser nula nem repetida. Geralmente é a coluna `id`. **FK (Foreign Key):** referencia a PK de outra tabela, criando o relacionamento. Ex: `leilao_id` na tabela `lances` garante integridade referencial (não é possível inserir um lance para um leilão inexistente).



Thymeleaf e Apresentação

10 perguntas

45. O que é o Thymeleaf e como ele funciona?

Motor de templates do lado do servidor. Processa arquivos HTML substituindo expressões `th:*` pelos dados reais vindos do Controller, e envia o HTML final já preenchido para o browser. O arquivo HTML é válido mesmo sem processamento (pode ser aberto no browser para preview), pois os atributos `th:*` são ignorados pelo browser.

46. O que significa `${...}` nos templates Thymeleaf?

Expressão de variável do Thymeleaf. `${leilao.nome}` acessa o atributo `nome` do objeto `leilao` no Model.

`#{numbers.formatCurrency(lance.valor)}` usa utilitário para formatar números.

`#{dates.format(leilao.dataInicio, 'dd/MM/yyyy')}` formata datas. O Thymeleaf substitui a expressão pelo valor calculado.

47. O que é `th:each` e como usamos para listas?

`th:each` é o equivalente ao `for`. Exemplo: `<tr th:each="leilao : ${leiloes}">` repete a linha para cada elemento da lista. Dentro do bloco, `${leilao.nome}` acessa o elemento atual. Também é possível obter o índice:

`th:each="item, stat : ${lista}"` onde `stat.index` é o índice e `stat.even/odd` a paridade.

48. O que é `th:if` e `th:unless`?

`th:if="${condicao}"` renderiza o elemento apenas se a condição for verdadeira. `th:unless="${condicao}"` é o contrário. Usamos para: exibir o banner de arremate só se houver lance vencedor, mostrar mensagens de erro quando o atributo `erro` está no Model, exibir lances apenas se o leilão for ABERTO.

49. O que é `th:text` e `th:href`?

`th:text="${valor}"` : substitui o conteúdo de texto do elemento, escapando automaticamente caracteres HTML

especiais (seguro contra XSS). `th:href="@{/leiloes/{id}(id=${leilao.id})}"` : gera URLs dinâmicas. A sintaxe

`@{...}` é a expressão de URL do Thymeleaf, resolvendo caminhos relativos ao contexto da aplicação.

50. O que é `th:action` e `th:object` em formulários?

`th:action="@{/lotes/novo}"` : define para onde o formulário envia os dados com URL dinâmica.

`th:object="${loteInput}"` : vincula o formulário ao objeto do Model, permitindo usar `th:field="*{nome}"` (com asterisco) como atalho. Juntos simplificam a criação e re-exibição de formulários com dados e erros.

51. O que é `th:replace` e como usamos o layout base?

`th:replace="~{layout/base :: conteudo}"` substitui o elemento pelo fragmento HTML chamado `conteudo` definido em `layout/base.html` com `th:fragment="conteudo"`. O `base.html` define cabeçalho, menu e rodapé comuns. Cada página fornece apenas o conteúdo específico, evitando duplicação de HTML.

52. Como exibimos erros de validação nos formulários Thymeleaf?

Usando `th:errors="*{campo}"` que exibe a mensagem de erro para o campo especificado, ou `#fields.hasErrors('campo')` para verificar se há erros. Quando o Controller retorna o formulário com erros no `BindingResult`, o Thymeleaf marca os campos inválidos e exibe as mensagens das anotações Bean Validation.

53. O que é a pasta static/ e o que colocamos nela?

`src/main/resources/static/` contém arquivos servidos diretamente pelo servidor: **CSS** (`static/css/style.css`), **JavaScript** (`static/js/`), **imagens** (`static/images/`). Acessados pela URL diretamente: `/css/style.css` . O Thymeleaf os referencia com `th:href="@{/css/style.css}"` .

54. Por que o Thymeleaf e não JSP ou outro motor de templates?

Thymeleaf é o template engine padrão do Spring Boot. Vantagens: HTML válido e visualizável no browser sem servidor (Natural Templates), integração nativa com Spring MVC e Bean Validation, suporte a layouts com `th:replace/insert` , mais moderno e ativo que JSP. Configurado automaticamente ao incluir a dependência no `pom.xml` .



Regras de Negócio e Domínio

10 perguntas

55. Quais são as regras para registrar um lance?

Todas as Specifications devem ser satisfeitas: (1) **LeilaoEmAndamentoSpec**: data/hora atual entre início e término; (2) **LanceAcimaDoMinimoSpec**: valor supera o preço mínimo do lote; (3) **LanceDeOutroUsuarioSpec**: usuário não pode ser o criador do leilão nem dono dos bens; (4) **LanceMaiorQueLanceAtualSpec**: em leilões ABERTOS, valor deve superar o maior lance existente.

56. Qual a diferença entre leilão ABERTO e FECHADO?

No leilão **ABERTO**, todos veem os lances dos outros e cada novo lance deve superar o maior atual (estilo pregão). No leilão **FECHADO**, os lances são sigilosos — ninguém vê o que os outros ofertaram; cada um envia apenas um lance e o maior ganha. A regra `LanceMaiorQueLanceAtualSpec` só se aplica aos leilões ABERTOS.

57. Como é determinado o lance vencedor?

O lance de maior valor entre todos os registrados. O `LanceRepository` usa SQL nativo: `SELECT * FROM lances WHERE leilao_id = ? ORDER BY valor DESC LIMIT 1`. O resultado é mapeado via `LanceVencedorProjection`. Exibido na tela somente quando o leilão está ENCERRADO (em leilões fechados os lances são sigilosos durante o andamento).

58. Como funciona o ciclo de vida de um leilão?

Três estados: (1) **AGUARDANDO**: criado mas ainda não chegou na data de início; (2) **EM_ANDAMENTO**: data/hora atual entre início e término, lances aceitos; (3) **ENCERRADO**: passou da data de término, não aceita lances e o vencedor pode ser revelado. O status é calculado dinamicamente no DTO `LeilaoResumo` comparando datas com `LocalDateTime.now()`.

59. O que é um Lote e como ele se relaciona com Leilão e Bens?

Um **Lote** é um conjunto de bens agrupados para ser leiloadado como uma unidade, com preço mínimo. Um Lote pertence a um Leilão (`@ManyToOne`). Um Lote tem múltiplos Bens (`@OneToMany` ou `@ManyToMany`). Cada Bem pertence a uma Categoria. Exemplo: um Lote pode conter "Notebook", "Mouse" e "Teclado" com preço mínimo de R\$ 2.000.

60. O que representa a entidade Usuario?

Representa qualquer pessoa (física ou jurídica) que participa do sistema — tanto como leiloeiro (cria leilões) quanto como comprador (faz lances). O campo de documento pode ser CPF (11 dígitos, pessoa física) ou CNPJ (14 dígitos, pessoa jurídica), identificado pelo comprimento ou por um campo `tipo` na entidade.

61. Por que um usuário não pode dar lance no próprio leilão?

Garantido pela `LanceDeOutroUsuarioSpec`: seria fraude o leiloeiro ou proprietário dos bens fazer lances para inflar artificialmente o preço. A Specification verifica se o `usuarioId` do lance é diferente do responsável pelo leilão e dos donos dos bens do lote, rejeitando o lance se for o mesmo.

62. O que é a Categoria e como se relaciona com os Bens?

`Categoria` é uma classificação dos bens (ex: Eletrônicos, Móveis, Veículos, Arte). Cada `Bem` pertence a exatamente uma `Categoria` (`@ManyToOne`). No banco, a tabela `bens` tem uma coluna `categoria_id` (FK) que referencia o `id` da tabela `categorias` .

63. O que acontece quando uma Specification não é satisfeita?

O `LanceService` chama `spec.isSatisfiedBy(lance)` . Se retornar `false` , lança a exceção correspondente (`LanceInvalidoException` , `LeilaoNaoEmAndamentoException` etc.). O `GlobalExceptionHandler` ou o `try-catch` no Controller captura, adiciona a mensagem ao Model e o Thymeleaf exibe via `th:if="{erro}"` .

64. Como a data e hora são tratadas no projeto?

Usamos `LocalDateTime` do Java 8+ para datas com hora sem fuso horário. No banco PostgreSQL, o tipo é `TIMESTAMP` . O Hibernate converte automaticamente. Para "está em andamento" usamos: `LocalDateTime.now().isAfter(inicio) && LocalDateTime.now().isBefore(fim)` . Nos formulários Thymeleaf usamos `input type="datetime-local"` .